Musician object ID as a parameter and checks if the authenticated user is either that musician or one of their bandmates. This relationship is found through the use of the user profile data object.

The steps for this view are as follows:

1. Use `get_object_or_404()` to turn the `musician_id` argument into a Musician object.
2. Check if the user profile object's `musician_profiles` field contains the argument musician (user is this musician).
3. If the user profile doesn't point to the argument musician, loop through all the bands associated with the argument musician and check if any of the user's musician profiles intersect with band membership.
4. If the user is neither the musician nor a bandmate, raise an error.

The code for these steps is provided in listing 6.6. Add it to the RiffMates/bands/views.py file, and then register a new `path` object in RiffMatest/bands/urls.py for the URL `musician_restricted`. Don't forget to add an expression for the `musician_id` argument, similar to how you did for the unrestricted version of this view in Chapter 4.

Listing 6.6 View with authorization code for musician profiles or band-mates

```
# RiffMates/bands/views.py
...
from django.http import Http404
...
@login_required
def musician_restricted(request, musician_id):
    musician = get_object_or_404(Musician, id=musician_id)
    profile = request.user.userprofile
    allowed = False

    if profile.musician_profiles.filter(
            id=musician_id).exists():
        allowed = True
    else:
        # User is not this musician, check if they're a band-mate
        musician_profiles = set(
            profile.musician_profiles.all()
        )
        for band in musician.band_set.all():
            band_musicians = set(band.musicians.all())
            if musician_profiles.intersection(
                    band_musicians):
                allowed = True
                break

    if not allowed:
        raise Http404("Permission denied")
```

Raise the Http404 exception for disallowed access.

request.user is the authenticated user, and .userprofile is the reverse relationship to the UserProfile ORM model.

Set to True if authorization succeeds.

Query the user's associated Musician objects for the argument musician_id. The .exists() query returns True if a matching relationship is found.

Store all the band's associated Musician objects as a set, and then intersect it with the user's musicians.

Store all the user's associated Musician objects as a set.

A nonempty intersection between the sets means one of the user's Musician objects was in the band's musician list

If allowed is not True, authorization is denied. Raise an error.